

Snowman Game - Rapport de Projet

1. Composition du Binôme

- Étudiant 1 : Ahmat Mahamat Ahmat (21 91 29 49)
- Étudiant 2 : Ahmed Zakaria Memar (22 21 17 64)

2. Introduction

Nous avons conçu une ontologie qui modélise de manière efficace un jeu de déplacement de bonhomme de neige et des boules. Les requêtes SPARQL sont utilisées pour récupérer et modifier l'état du jeu.

Le jeu est fonctionnel et gère correctement les bords de la grille.

3. Concepts ajoutés à l'ontologie

- **CellBall**
 - **Définition** : Représente une cellule contenant un bonhomme de neige.
 - **Utilité** : Permet de définir les cellules qui ont une boule de neige pour faciliter les déplacements et les fusions.
- **CellFree**
 - **Définition** : Une cellule libre qui ne contient pas de Snowman.
 - **Utilité** : Permet d'identifier les cellules accessibles pour le joueur et pour le déplacement des boules de neige.
- **CellNorthPlayer, CellSouthPlayer, CellEastPlayer, CellWestPlayer**
 - **Définition** : Classes représentant les cellules situées respectivement **au nord, au sud, à l'est et à l'ouest du joueur**.
 - **Utilité** : Ces concepts permettent de **faciliter les requêtes SPARQL** en identifiant les cellules adjacentes occupées par le joueur.
 - **Équivalence** :
 - Une **CellNorthPlayer** est une cellule qui possède une cellule **CellPlayer** au **sud** (`hasSouth`).
 - Une **CellSouthPlayer** est une cellule qui possède une cellule **CellPlayer** au **nord** (`hasNorth`).
 - Une **CellEastPlayer** est une cellule qui possède une cellule **CellPlayer** à **l'ouest** (`hasWest`).
 - Une **CellWestPlayer** est une cellule qui possède une cellule **CellPlayer** à **l'est** (`hasEast`).
- **MovableToNorth, MovableToSouth, MovableToEast, MovableToWest**
 - **Définition** : Définit si une cellule contenant un Snowman peut être déplacée dans une direction spécifique.
 - **Utilité** : Permet de vérifier si une boule de neige peut être poussée par le joueur dans une direction donnée.
 - **Équivalence** :
 - Une cellule **MovableToNorth** est une **CellBall** qui possède une **CellFree** au **nord** (`hasNorth`).
 - Une cellule **MovableToSouth** est une **CellBall** qui possède une **CellFree** au **sud** (`hasSouth`).
 - Une cellule **MovableToEast** est une **CellBall** qui possède une **CellFree** à **l'est** (`hasEast`).
 - Une cellule **MovableToWest** est une **CellBall** qui possède une **CellFree** à **l'ouest** (`hasWest`).
- **littleAndBigSnowman**
 - **Définition** : Classe représentant l'assemblage d'un petit et d'un grand bonhomme de neige.
 - **Utilité** : Permet d'organiser la fusion des boules de neige en différentes étapes jusqu'à obtenir le bonhomme de neige final.

4. Propriétés ajoutées à l'ontologie

- **hasSnowman**
 - **Définition** : Relation indiquant qu'une cellule possède un bonhomme de neige (`Snowman`).
 - **Utilité** : Utilisée pour identifier les cellules contenant une boule de neige et vérifier si elles peuvent être déplacées ou fusionnées.
- **hasNorth, hasSouth, hasEast, hasWest**
 - **Définition** : Représente les relations de voisinage entre cellules de la grille.
 - **Utilité** : Ces propriétés sont utilisées pour **naviguer sur la grille**, vérifier la position des éléments et déterminer si des déplacements sont possibles.

4.1. Pourquoi ces ajouts ?

- Ces ajouts permettent **d'améliorer la navigation** sur la grille en identifiant facilement les cellules autour du joueur.
- Les concepts **MovableToX** et **CellFree** facilitent la **vérification des déplacements** des Snowmen.
- Les classes **littleAndBigSnowman** et autres variantes aident à gérer la **fusion progressive des Snowmen**.
- L'utilisation des relations **hasNorth, hasSouth, etc.** facilite les requêtes SPARQL en évitant d'écrire manuellement des relations de voisinage.

4.2. Conclusion

L'intérêt de ces équivalences est d'optimiser les requêtes en permettant d'interroger directement des concepts abstraits au lieu de vérifier chaque condition séparément.

Cependant, l'inférence pour **MovableToX** et **CellFree** ne fonctionne pas correctement, nous avons donc dû adapter nos requêtes pour contourner ce problème.

5. Requêtes SPARQL Utilisées

5.1. Récupérer l'état du jeu

```
SELECT ?player ?north ?south ?east ?west ?littleSnowman ?mediumSnowman ?bigSnowman ?littleAndMediumSnowman
?mediumAndBigSnowman ?littleAndBigSnowman ?finalSnowman
WHERE {
  ?player a :CellPlayer.
  OPTIONAL { ?player :hasNorth ?north . }
  OPTIONAL { ?player :hasSouth ?south . }
  OPTIONAL { ?player :hasEast ?east . }
  OPTIONAL { ?player :hasWest ?west . }
  OPTIONAL { ?littleSnowman :hasSnowman :littleSnowman . }
  OPTIONAL { ?mediumSnowman :hasSnowman :mediumSnowman . }
  OPTIONAL { ?bigSnowman :hasSnowman :bigSnowman . }
  OPTIONAL { ?littleAndBigSnowman :hasSnowman :littleAndBigSnowman . }
  OPTIONAL { ?littleAndMediumSnowman :hasSnowman :littleAndMediumSnowman . }
  OPTIONAL { ?mediumAndBigSnowman :hasSnowman :mediumAndBigSnowman . }
  OPTIONAL { ?finalSnowman :hasSnowman :finalSnowman . }
}
```

Objectif : Récupérer l'état du jeu, y compris la position du joueur et des différents types de snowmen.

5.2. Déplacer le joueur

```
DELETE {
  ?oldCell a :CellPlayer .
}
INSERT {
  :${destination} a :CellPlayer .
}
WHERE {
  ?oldCell a :CellPlayer .
}
```

Objectif : Met à jour la position du joueur lorsqu'il se déplace sur une nouvelle cellule.

5.3. Réinitialiser le jeu

```
DELETE WHERE {
  ?player a :CellPlayer .
  ?cell :hasSnowman ?snowman .
};

INSERT DATA {
  :cell54 a :CellPlayer .
  :cell25 :hasSnowman :littleSnowman .
  :cell68 :hasSnowman :mediumSnowman .
  :cell82 :hasSnowman :bigSnowman .
}
```

Objectif : Supprime toutes les entités du jeu et les remet dans leur position initiale.

5.4. Vérifier si le déplacement est possible

```
SELECT ?newCell ?snowman ?nextCell ?nextSnowman
WHERE {
  ?player a :CellPlayer .
  ?player :${direction} ?newCell .
  OPTIONAL { ?newCell :hasSnowman ?snowman . }
  OPTIONAL {
    ?newCell :${direction} ?nextCell .
    OPTIONAL { ?nextCell :hasSnowman ?nextSnowman . }
  }
}
```

Objectif : Vérifie si le joueur peut se déplacer dans une direction donnée, s'il y a un snowman et si celui-ci peut être poussé.

5.5. Déplacement et poussée des Snowmen

```

DELETE {
  ?player a :CellPlayer .
  :${newCell} :hasSnowman :${snowman}
}
INSERT {
  :${newCell} a :CellPlayer .
  :${nextCell} :hasSnowman :${snowman}
}
WHERE {
  ?player a :CellPlayer .
  :${newCell} :hasSnowman :${snowman}
}

```

Objectif : Déplace le joueur et pousse un snowman si possible.

5.6. Fusionner deux snowmen

```

DELETE {
  :${cellA} :hasSnowman :${snowmanA} .
  :${cellB} :hasSnowman :${snowmanB}
}
INSERT { :${cellB} :hasSnowman :${newSnowman} }
WHERE {
  :${cellA} :hasSnowman :${snowmanA} .
  :${cellB} :hasSnowman :${snowmanB}
}

```

Objectif : Fusionne deux snowmen lorsqu'ils se rencontrent selon des règles précises.